

ITA0611 - Machine Learning

Assignment Questions

Name:	Kota Pujitha
Reg. :	192311124
Course Code & Name	ITA0611 - Machine Learning
Assignment Title	Design and Development of an Intelligent Early-Warning System for Crop Disease and Yield Risk Prediction Using Machine Learning
Course Outcome(s) - CO	CO6 Instance-based learning including KNN, LWR and case-based reasoning CO7 Data manipulation and analysis using NumPy and Pandas Data cleaning, statistical analysis, aggregation and visualization CO8 Integration and application of theoretical and practical ML knowledge
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger; SDG 9 - Industry, Innovation and Infrastructure; SDG 13Climate Action

Problem Statement

Farmers often face significant losses due to crop diseases, unfavorable weather conditions, and variations in soil and environmental parameters. Develop an **intelligent machine learning–based early-warning system** that can analyze agricultural data such as temperature, rainfall, humidity, soil moisture, soil nutrients, crop type, disease symptoms, and historical yield to predict **crop disease risk and potential yield loss**. The system should preprocess and analyze the dataset using **NumPy and Pandas**, identify relevant patterns, and implement suitable machine learning approaches such as **Bayesian classification, k-nearest neighbors, locally weighted regression, decision trees, and neural networks**. Students should investigate how different learning approaches represent and search the hypothesis space and compare their predictive performance. A suitable **multilayer perceptron with back propagation** may be developed for risk prediction, while an appropriate **genetic algorithm/genetic programming approach** can be explored for feature or model optimization. The final system should provide interpretable predictions, performance evaluation, visualizations, and recommendations that can support farmers in taking timely preventive or corrective actions.

Question 1 – Data Collection and Preprocessing

The assignment requires a suitable crop-yield and disease dataset and the use of NumPy and Pandas for cleaning, preprocessing, transformation and analysis.

Answer

Collect records containing weather, soil, crop, disease-symptom and yield information.

Load the CSV file into a Pandas DataFrame.

Inspect data types, missing values, duplicates and summary statistics.

Replace or remove missing values using suitable numerical and categorical strategies.

Remove duplicate records and correct invalid or impossible values.

Encode categorical variables such as crop type and symptom categories.

Scale numerical features for distance-based KNN, LWR and neural-network models.

Separate predictors X from target y and split the data into training and testing sets.

Use exploratory statistics and visualizations to identify patterns and outliers.

Keep preprocessing consistent between training and future prediction data.

Python Implementation

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
df = pd.read_csv("crop_risk.csv")
```

```
print(df.head())
```

```
print(df.info())
```

```
print(df.isnull().sum())
```

```
df = df.drop_duplicates()
```

```
numeric_cols = ["Temperature", "Rainfall", "Humidity", "Soil_Moisture",  
                "Nitrogen", "Phosphorus", "Potassium", "Historical_Yield"]
```

```
categorical_cols = ["Crop_Type", "Disease_Symptoms"]
```

```
for col in numeric_cols:
```

```
    df[col] = df[col].fillna(df[col].median())
```

```

for col in categorical_cols:

    df[col] = df[col].fillna(df[col].mode()[0])

X = df[numeric_cols + categorical_cols]

y = df["Disease_Risk"]

preprocess = ColumnTransformer([

    ("num", StandardScaler(), numeric_cols),

    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)

])

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.20, random_state=42, stratify=y

)

```

Question 2 – Concept Learning and Version Space

Answer

Crop disease-risk prediction can be represented as a concept-learning problem. A hypothesis describes conditions under which a crop is classified as high disease risk.

Element	Definition
Instance	A crop observation described by environmental, soil and symptom attributes.
Target concept	Whether the crop observation belongs to the selected risk class, e.g., High Disease Risk.
Positive example	An instance labelled High disease risk.
Negative example	An instance labelled not High disease risk.
Hypothesis space	All allowed combinations/generalizations of the selected attributes.

Inductive bias	Assumption that similar environmental and crop conditions tend to have similar disease-risk outcomes.
----------------	---

Manual Candidate-Elimination Example

Temperature	Humidity	Rainfall	Symptoms	Class
High	High	High	Severe	Yes
High	High	High	Mild	Yes
Moderate	High	High	Severe	No
High	High	Moderate	Severe	Yes

Using the first positive example, the specific boundary begins as $\langle \text{High, High, High, Severe} \rangle$. After the second positive example, Symptoms is generalized because it differs while the remaining attributes match.

Stage	S boundary	G boundary
Initial	$\langle \emptyset, \emptyset, \emptyset, \emptyset \rangle$	$\langle ?, ?, ?, ? \rangle$
After positive 1	$\langle \text{High, High, High, Severe} \rangle$	$\langle ?, ?, ?, ? \rangle$
After positive 2	$\langle \text{High, High, High, ?} \rangle$	$\langle ?, ?, ?, ? \rangle$
After negative 3	$\langle \text{High, High, High, ?} \rangle$	General hypotheses excluding the negative instance
After positive 4	$\langle \text{High, High, ?, ?} \rangle$	Consistent generalizations

The exact final Version Space depends on the complete training table and attribute domains. The important idea is that Candidate Elimination removes hypotheses inconsistent with observed positive and negative examples.

Question 3 – Decision Tree Learning

Answer

A Decision Tree recursively divides the data into groups using the attribute that gives the best class separation.

Gini Index, Information Gain or Gain Ratio can be used for attribute selection.

For a node, Gini impurity is $\text{Gini}(S) = 1 - \sum p_i^2$.

Information Gain is $\text{IG}(S,A) = \text{Entropy}(S) - \sum(|S_v|/|S|)\text{Entropy}(S_v)$.

A low-Gini or high-information-gain split is preferred.

The resulting paths can be interpreted as decision rules.

Example rule: if symptoms are severe and humidity is high, predicted disease risk can be High.

Tree depth and minimum-sample parameters should be controlled to reduce overfitting.

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, classification_report
```

```
tree = Pipeline([
    ("prep", preprocess),
    ("model", DecisionTreeClassifier(
        criterion="gini", max_depth=5, random_state=42
    ))
])
tree.fit(X_train, y_train)
pred_tree = tree.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, pred_tree))
print(classification_report(y_test, pred_tree))
```

Question 4 – Bayesian Classification

Answer

Bayesian classification calculates the probability of a class given the observed attributes. Bayes' theorem is:

$$P(C | X) = P(X | C) P(C) / P(X)$$

C represents a disease-risk class.

X represents the observed feature vector.

P(C) is the prior probability of the class.

P(X|C) is the likelihood of observing the features for that class.

P(C|X) is the posterior probability used for prediction.

Maximum Likelihood Estimation (MLE) estimates probabilities/parameters from the training observations.

For continuous features, Gaussian Naive Bayes can estimate class-wise mean and variance.

The class with the largest posterior probability is selected.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Use numeric-only features after median imputation for a simple Gaussian NB example.
X_num = df[numeric_cols].values
y_nb = df["Disease_Risk"].values

Xtr, Xte, ytr, yte = train_test_split(
    X_num, y_nb, test_size=0.20, random_state=42, stratify=y_nb
)
nb = Pipeline([
    ("scale", StandardScaler()),
    ("model", GaussianNB())
])
nb.fit(Xtr, ytr)
pred_nb = nb.predict(Xte)
print("Accuracy:", accuracy_score(yte, pred_nb))
print(classification_report(yte, pred_nb))
```

Question 5 – K-Nearest Neighbors

Answer

KNN is an instance-based learning method that stores training examples rather than learning a fixed parametric equation.

For a new crop observation, distances to training examples are calculated.

The K nearest examples vote for the predicted risk class.

Feature scaling is important because distance calculations are affected by feature magnitude.

Small K can be sensitive to noise, while very large K can oversmooth class boundaries.

Different K values should be tested using validation accuracy or F1-score.

A validation curve can be plotted to select a suitable K.

KNN is simple and interpretable at the instance level but can become slower as the training set grows.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score

results = []
for k in range(1, 16):
    knn = Pipeline([
```

```

("prep", preprocess),
("model", KNeighborsClassifier(n_neighbors=k))
])
knn.fit(X_train, y_train)
p = knn.predict(X_test)
results.append((k, accuracy_score(y_test, p), f1_score(y_test, p, average="weighted")))

for row in results:
    print(row)

```

Question 6 – Locally Weighted Regression

Answer

Locally Weighted Regression (LWR) predicts a target by giving greater importance to training examples that are closer to the query point. A common Gaussian weight is:

$$w_i = \exp(-\|x - x_i\|^2 / (2\tau^2))$$

x is the query observation and x_i is a training observation.

τ (tau) is the weighting/bandwidth parameter.

Small τ creates a very local model and may have high variance.

Large τ produces smoother predictions and may increase bias.

The weighted regression parameters are estimated around each query point.

MAE and RMSE are suitable metrics for yield prediction.

The best τ should be selected using validation performance.

LWR can model local nonlinear behaviour but is computationally expensive for large datasets.

```

import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error

```

```

# Simple LWR from first principles for numeric features

```

```

def lwr_predict(X_train, y_train, X_query, tau):

```

```

    preds = []

```

```

    X_train = np.asarray(X_train, dtype=float)

```

```

    y_train = np.asarray(y_train, dtype=float)

```

```

    X_query = np.asarray(X_query, dtype=float)

```

```

    Xb = np.c_[np.ones(len(X_train)), X_train]

```

```

    for x in X_query:

```

```

        xb = np.r_[1.0, x]

```

```

        d2 = np.sum((X_train - xb) ** 2, axis=1)

```

```

w = np.exp(-d2 / (2 * tau ** 2))
W = np.diag(w + 1e-8)
theta = np.linalg.pinv(Xb.T @ W @ Xb) @ (Xb.T @ W @ y_train)
preds.append(xb @ theta)
return np.array(preds)

yield_features = ["Temperature", "Rainfall", "Humidity", "Soil_Moisture",
                  "Nitrogen", "Phosphorus", "Potassium", "Historical_Yield"]

Xy = df[yield_features].fillna(df[yield_features].median())
yy = df["Yield"].fillna(df["Yield"].median())

Xa, Xb, ya, yb = train_test_split(
    Xy.values, yy.values, test_size=0.20, random_state=42
)
scaler = StandardScaler()
Xa_s = scaler.fit_transform(Xa)
Xb_s = scaler.transform(Xb)

for tau in [0.1, 0.3, 0.5, 1.0, 2.0]:
    pred = lwr_predict(Xa_s, ya, Xb_s, tau)
    mae = mean_absolute_error(yb, pred)
    rmse = np.sqrt(mean_squared_error(yb, pred))
    print("tau =", tau, "MAE =", mae, "RMSE =", rmse)

```

Question 7 – Neural Network and Back Propagation

Answer

The Multilayer Perceptron (MLP) contains an input layer, one or more hidden layers and an output layer.

Input features are transformed through weighted connections and activation functions.

Forward propagation generates a predicted class.

The loss function measures the difference between prediction and target.

Back propagation computes gradients of the loss with respect to model parameters.

Weights are updated using an optimization method such as gradient descent.

Hidden-layer size, learning rate, activation, batch size and training iterations affect performance.

Validation performance should be monitored to reduce overfitting.

```
from sklearn.neural_network import MLPClassifier
```

```
mlp = Pipeline([
```



```

("prep", preprocess),
("model", MLPClassifier(
hidden_layer_sizes=(32, 16),
activation="relu",
solver="adam",
learning_rate_init=0.001,
max_iter=500,
random_state=42
))
])
mlp.fit(X_train, y_train)
pred_mlp = mlp.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred_mlp))
print(classification_report(y_test, pred_mlp))

```

Question 8 – Genetic Algorithm / Genetic Programming

Answer

A Genetic Algorithm can select useful features or optimize model hyperparameters.

Each chromosome represents one candidate solution.

For feature selection, a binary gene indicates whether a feature is selected (1) or rejected (0).

The fitness function should reward predictive performance while discouraging unnecessarily large feature sets.

Selection chooses stronger chromosomes for reproduction.

Crossover combines parts of two parent chromosomes.

Mutation randomly changes a small number of genes to maintain diversity.

The process continues for multiple generations until a stopping condition is reached.

Example fitness function:

Fitness = Validation_F1 – $\lambda \times (\text{Number_of_Selected_Features} / \text{Total_Features})$

Simplified GA feature-selection example

```
import random
```

```
def fitness(mask, X, y):
```

```
    selected = np.where(np.array(mask) == 1)[0]
```

```
    if len(selected) == 0:
```

```
        return 0
```

```
    Xs = X[:, selected]
```

```
    xa, xb, ya, yb = train_test_split(
```

```
        Xs, y, test_size=0.20, random_state=42, stratify=y
```

```
)
model = DecisionTreeClassifier(max_depth=5, random_state=42)
model.fit(xa, ya)
score = f1_score(yb, model.predict(xb), average="weighted")
penalty = 0.01 * len(selected) / X.shape[1]
return score - penalty
```

```
# In a full implementation, maintain a population, select parents,
# apply crossover/mutation, evaluate fitness, and retain the best chromosome.
```

The improvement is evaluated by comparing the optimized model with the baseline model using the same validation protocol. The GA should only be considered useful if it improves validation performance or reduces model complexity without harming generalization.

Question 9 – Model Evaluation and Comparison

Answer

Accuracy = correctly classified observations / total observations.

Precision measures how many predicted positive cases are actually positive.

Recall measures how many actual positive cases are successfully detected.

F1-score is the harmonic mean of precision and recall.

MAE measures average absolute regression error.

RMSE gives greater penalty to larger regression errors.

Interpretability, scalability and generalization should be considered in addition to raw accuracy.

For disease early warning, recall can be particularly important because missing a high-risk case may lead to delayed action.

Model	Primary Task	Strength	Limitation
Decision Tree	Classification	Easy-to-understand rules	Can overfit without control
Bayesian Classifier	Classification	Fast and probabilistic	Distribution/independence assumptions may limit accuracy
KNN	Classification	Simple, instance-based	Prediction can be slow for large datasets
LWR	Regression	Captures local relationships	Computationally expensive
MLP	Classification	Learns complex	Less interpretable and

		nonlinear patterns	needs parameter tuning
GA + ML	Optimization	Can select features/tune models	Adds computational cost

Evaluation Code

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
def classification_metrics(y_true, y_pred):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, average="weighted", zero_division=0),
        "Recall": recall_score(y_true, y_pred, average="weighted", zero_division=0),
        "F1": f1_score(y_true, y_pred, average="weighted", zero_division=0)
    }
```

```
print("Decision Tree:", classification_metrics(y_test, pred_tree))
print("Naive Bayes:", classification_metrics(yte, pred_nb))
print("MLP:", classification_metrics(y_test, pred_mlp))
```

Because the supplied assignment does not contain actual dataset observations or experimental outputs, numerical performance values should be generated by running the code on the chosen dataset. It would be inappropriate to invent accuracy, MAE or RMSE values.

Question 10 – Integrated Crop Risk Early-Warning System

System Design

Data input module receives current environmental, soil, crop and symptom information.

Preprocessing module performs validation, missing-value handling, encoding and scaling.

Disease-risk model predicts Low, Medium or High disease risk.

Yield-risk model predicts the potential level of yield loss.

Risk fusion module combines disease and yield predictions into an overall warning level.

Visualization module displays trends, risk categories, feature patterns and model results.

Recommendation module converts risk levels into timely preventive or corrective actions.

The system stores prediction time, inputs, model output and recommendation for monitoring.

Workflow

Data Collection → Data Cleaning → Feature Engineering → Train/Test Split → Model Training → Validation → Model Selection → Risk Prediction → Visualization → Recommendation → Early Warning

Example Recommendation Logic

Risk	Example Recommendation
Low	Continue normal monitoring and routine crop management.
Medium	Increase monitoring frequency, inspect affected areas and review irrigation/nutrient conditions.
High	Inspect the crop immediately, isolate affected areas where appropriate, review environmental conditions and consult an agricultural expert before corrective treatment.

The recommendations are decision-support suggestions rather than guaranteed diagnoses. Actual intervention should consider crop type, local agronomic conditions, approved practices and expert advice.

Five Meaningful Visualizations

Correlation heatmap of numerical environmental, soil and yield variables.

Distribution plots for temperature, rainfall, humidity and soil moisture.

Disease-risk class distribution showing Low, Medium and High counts.

KNN validation curve showing performance against different K values.

Model comparison chart showing accuracy/F1 for classification models and MAE/RMSE for yield regression.

One-Page Reflection

The proposed early-warning system demonstrates how machine learning can support agricultural decision-making by combining data preprocessing, instance-based learning, probabilistic learning, tree models, regression, neural networks and evolutionary optimization. A major limitation is that prediction quality depends strongly on the quality, representativeness and timeliness of agricultural data. Missing observations, sensor errors, changing weather patterns and regional differences can reduce generalization. The system may also show bias if some crops, regions or disease conditions are underrepresented in the training data.

The bias–variance trade-off is important. A very flexible model such as a deep tree or highly local LWR can have low training error but high variance, while an overly simple model may have high bias. KNN is also sensitive to K: small K values can react strongly to noise, whereas large K values may miss local patterns. Validation, cross-validation, feature scaling and appropriate regularization can improve generalization.

The system supports SDG 2 – Zero Hunger by helping identify crop disease and yield risks earlier, potentially reducing preventable losses. It also relates to SDG 13 – Climate Action because weather and environmental changes are incorporated into risk analysis. SDG 9 is relevant because the project applies digital innovation and intelligent infrastructure to agricultural

monitoring.

Future improvements include collecting larger multi-season datasets, incorporating satellite imagery and soil sensors, using explainable-AI methods, calibrating probabilities, performing continuous model monitoring and retraining models as new field data become available. The system should be evaluated in real agricultural settings before being relied upon for operational decisions.

Conclusion

The assignment provides a complete design for an intelligent crop disease and yield-risk early-warning system. Pandas and NumPy support data preparation and analysis, while Candidate Elimination explains concept learning and hypothesis-space search. Decision Tree and Bayesian models provide interpretable classification approaches, KNN provides instance-based classification, and LWR provides local regression. MLP with back propagation can model complex nonlinear patterns, while a Genetic Algorithm can optimize feature selection or model parameters. The final integrated system compares these approaches using suitable evaluation metrics and converts predictions into practical early-warning recommendations.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Assessment Criteria	Marks	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Data Collection, Cleaning & Preprocessing	10	Complete and accurate preprocessing with appropriate justification	Mostly correct preprocessing	Basic preprocessing with some omissions	Major preprocessing errors
Statistical Analysis & Visualization	10	Comprehensive analysis with meaningful visualizations and interpretations	Good analysis and visualizations	Basic analysis	Limited/inappropriate analysis
Concept Learning, Version Space & Inductive Bias	10	Correct hypothesis representation and clear analysis of Version Space/Candidate Elimination	Mostly correct with minor gaps	Partial understanding	Incorrect/incomplete implementation
Decision Tree & Concept Representation	10	Correct tree construction, attribute selection and interpretation	Correct implementation with minor issues	Basic implementation	Poor tree construction/interpretation
Bayesian Learning & MLE	10	Correct Bayesian formulation, probability calculation and MLE application	Mostly correct implementation	Basic implementation	Significant conceptual errors
KNN & LWR Implementation	10	Correct implementation with parameter analysis and interpretation	Good implementation	Basic implementation	Incorrect/limited implementation
MLP & Back Propagation	15	Correct architecture, training, back propagation and parameter analysis	Good implementation with minor limitations	Basic neural-network implementation	Major implementation/conceptual issues
Genetic Algorithm / Genetic Programming	10	Appropriate representation, fitness function and evolutionary optimization	Correct implementation with minor gaps	Basic GA implementation	Poor/incomplete implementation

Model Evaluation & Comparative Analysis	10	Comprehensive comparison with appropriate metrics and strong justification	Good comparison and justification	Limited comparison	Inadequate evaluation
Integrated System,	5	Complete working	Functional solution with	Partially integrated	Incomplete/non-functional
Interpretation & Presentation		solution with meaningful recommendations and excellent presentation	good presentation	solution	solution
Total	100				